Mobile Programming

Java Question Bank

Section A: Theoretical/Conceptual Questions

1. Differentiate between HashMap and ConcurrentHashMap.
2. Is IOException Checked or Unchecked?
3. Explain Overriding in Java.
4. Explain Stack indexOf() method in Java with an example.
5. What is the impact on the order of elements in an array of integers when using the remove() method of the Integer class?
6. Difference between object class equals method and string class equals method.
7. Explain @configuration annotation.
8. Explain Hash collision.
9. What is ConcurrentModificationException and how can we remove that from list and any other collection interface?
10. How to implement universal search using JPA?
11. What are the different bean scopes available in the Spring Framework and how do they differ from each other?
12. What are some of the new features introduced in Java 8 and how do they enhance Java programming?
13. What are the advantages of using the Optional interface in Java and how does it help in writing cleaner and more robust code?
14. What are Java Streams and how do they provide a more functional approach to processing collections of data in Java?
15. What are some commonly used Spring annotations in Java and how do they simplify the process of configuring and working with Spring Framework components?
16. What are Hibernate mappings and how do they define the relationship between Java objects and database tables, allowing for efficient and flexible database access in Java applications?
17. What agile methodology do you follow in software development and how does it help you to deliver high-quality software products efficiently and effectively?
18. What is the Agile methodology in software development and how does it differ from traditional software development approaches in terms of project management, team collaboration, and product delivery?

19. If you, as a software developer, are assigned a front-end ticket, how comfortable are you working on it, and what steps would you take to ensure that the front-end task is completed efficiently and to a high standard?
20. What is a HashMap in Java and how does it work, and what are some of the key advantages and disadvantages of using a HashMap in your Java applications?
21. What is an immutable class in Java and how does it differ from a mutable class, and what are some benefits and drawbacks of using immutable classes in Java programming?
22. What is a HashTable in Java and how does it differ from other Map implementations, and what are some key features and use cases for HashTable in Java programming?
23. Is it possible to store a null key or a null value in a HashMap in Java, and what are some potential issues that can arise when working with null values in a HashMap?
24. What is unit testing in software development, and how can you use assertion statements to test individual units of code in Java, and what are some key benefits of performing unit testing in Java programming?
25. What is the Builder Pattern in Java, and how does it help to simplify the creation of complex objects by separating the construction of the object from its representation, and what are some advantages and use cases of using the Builder Pattern in Java programming?
26. Explain Exception handling in Java.
27. Explain OOPs.
28. Explain Garbage Collection.
29. What is an interface in Java and how does it provide a mechanism for defining a set of methods that a class must implement, and what are some key features and use cases of using interfaces in Java programming?
30. What is the difference between a class and an object in Java, and how do they relate to each other in object-oriented programming, and what are some key features and use cases of classes and objects in Java programming?
31. What are some common design patterns in Java, and how do they provide a reusable and optimized solution to common software design problems, and what are some key advantages and use cases of using design patterns in Java programming?
32. What is the underlying data structure used in implementing the HashMap in Java, and how can you implement your own version of the HashMap data structure using an array and linked lists, and what are some key considerations and challenges in implementing a custom HashMap in Java programming?
33. How many classes are in the statement "Passenger was asked for a Ticket by the Conductor"?
34. What is an AOT compiler?
35. Explain @Autowired annotation and the types of it.
36. What is the Singleton design pattern in Java, and how does the Double-Checked Locking implementation technique work to ensure that only one instance of the Singleton class is created in a multithreaded environment? Can you provide an example of a Java class that implements the Singleton pattern using the Double-Checked Locking technique, and what are some potential drawbacks and limitations of using this technique in practice?
37. Why is String used as a key in HashMap? Can you explain the reason behind this design decision and its advantages? How does it affect the performance of HashMap compared to other data structures? Can you provide an example implementation that uses String as a key in HashMap and explain how it works?
38. Explain Encapsulation.

39. **How can you handle unchecked exceptions in Java using try-catch blocks? Can you provide an example?**
40. **What is time complexity in Java and how can you determine it for a given code? Provide an example of a Java code and explain how to calculate its time complexity.**
41. **Could you explain what is the purpose of the main method in Java and briefly describe the OOPs principles?**
42. **Explain Multithreading with proper examples.**
43. **Explain the differences between List, Set, and Map in Java Collections framework. Provide examples of scenarios where you would use each of these data structures.**
44. **Suppose you have a web application that allows users to create, read, update, and delete information about different types of animals. Design a REST API using a Spring MVC controller that provides endpoints for these CRUD operations.**

    Your API should support the following endpoints:

    GET /animals - retrieves a list of all animals
    GET /animals/{id} - retrieves a specific animal by its ID
    POST /animals - creates a new animal
    PUT /animals/{id} - updates an existing animal by its ID
    DELETE /animals/{id} - deletes an animal by its ID
    The animal object should have the following properties:

    id (unique identifier)
    name
    species
    age
    description

    Make sure your controller returns responses in JSON format and handles appropriate HTTP status codes for each operation.
45. **Explain Abstraction with a proper example.**
46. **Explain Data Hiding with a proper example.**
47. **What is the purpose of the main method in Java? Can we define a main method with a return type of int instead of void? If so, what is the significance of passing an int array as an argument in the main method?**
48. **Explain Polymorphism with proper examples.**
49. **Difference between HashMap and TreeMap.**
50. **What is the difference between Anonymous inner class and Local inner class in Java? Provide examples to illustrate your answer.**
51. **What is the difference between Controller and RestController?**
52. **Explain the life cycle of a Spring bean in detail.**
53. **Write a Java singleton class that is thread-safe using the double-checked locking technique. The class should have a private constructor, a private static instance, and a public static method that returns the instance. Also, explain the double-checked locking technique.**
54. **What are the differences between Factory Design Pattern and Singleton Design Pattern?**
55. **What is Spring Boot and how does it work with databases? Can you explain the steps to configure a database in a Spring Boot application?**

56. **Explain the working of JPA Repository in Spring Boot framework.**
57. **What is a deployment tool in software development?**
58. **What is the process for deploying code to different environments?**
    a. **Explain the steps involved in deploying code to development, staging, and production environments.**
    b. **What are the best practices for deploying code to each environment?**
    c. **What tools can be used to automate the deployment process?**
    d. **How can version control systems be integrated into the deployment process?**
    e. **What are the common challenges faced during the deployment process and how can they be mitigated?**
59. **Write a Java class named DuplicateChecker containing a method containsDuplicate that takes a String as input and returns a boolean indicating whether the input String contains any duplicate characters. Write JUnit tests to verify the functionality of the containsDuplicate method.**
60. **Define Nested class.**
61. **Differentiate between JVM and JDK.**
62. **Explain synchronization in Java.**
63. **What are sorting algorithms and how do they work in computer science? Can you explain some commonly used sorting algorithms, such as bubble sort, insertion sort, quicksort, and merge sort? What are the differences between these sorting algorithms in terms of their time complexity, space complexity, stability, and suitability for different types of data? How do you choose the most appropriate sorting algorithm for a given scenario? Can you provide some examples of real-world applications that use sorting algorithms, such as database management systems, search engines, and data analysis tools?**
64. **What is the Chrome Extension API, and how does it enable developers to create browser extensions for the Chrome browser?**
65. **What are the different approaches to bundling and deploying desktop applications for different operating systems, such as Windows, macOS, and Linux? How do developers package their applications into installable bundles or executables that can be easily installed and run on users' machines?**
66. **Can you explain the data flow in your project and how it enables the system to collect, process, and store data from various sources?**
67. **Differentiate between StringBuffer and StringBuilder.**
68. **What is Thread safe in Java?**
69. **How to achieve thread safety in Java?**
70. **What are the changes done in the new version of Java in terms of garbage collection?**
71. **What is a Hystrix Circuit Breaker?**
72. **How do you specify the application port in a Spring Boot application?**
73. **How to change the default port number in a Java application?**
74. **Explain Spring Boot Actuator.**

Section B: Coding Questions

75. **How can you count the number of occurrences of a particular character in a Java String, and what are some different approaches that can be used to achieve this?**
76. **What is a possible approach to find the most frequently occurring character in a Java String, and how can you implement this solution in Java programming?**

77. **What is an approach to add the individual digits of a given integer in Java programming, and how can you implement this solution to calculate the sum of digits for an integer such as 567 and return the result of 9?**
78. **Write a Java program to find the largest subarray in an integer array that has a sum of 0. Can you explain your approach to solving this problem and provide a step-by-step explanation of your code? How does your program handle edge cases, such as when there is no subarray with a sum of 0 or when the input array is empty?**

    Eg: Input = { 3, 4, -7, 3, 1, 3, 1, -4, -2, -2 }

    { 3, 4, -7 }
    { 4, -7, 3 }
    { -7, 3, 1, 3 }
    { 3, 1, -4 }
    { 3, 1, 3, 1, -4, -2, -2 }
    { 3, 4, -7, 3, 1, 3, 1, -4, -2, -2 } // This should be the output
79. **Can you provide a program in Java that finds all the subarrays in an array whose sum is zero? How does your implementation work? Can you explain the time and space complexity of your solution? Are there any optimizations that can be made to improve the efficiency of your program?**
80. **Write a Java program to find the sum of all the numbers in a given string. For example, if the input string is "1ab2d4hj6", the program should output the sum of numbers in the string, which is 13.**

    The program should take a string as input from the user and use regular expressions to extract all the numbers from the string. It should then iterate over the extracted numbers and add them up to get the final sum. If there are no numbers in the string, the program should output 0.
81. **Write a Java program that takes two strings as input and finds the number of occurrences of the second string in the first string.**

    Example:
    Input:
    String 1: "hello world"
    String 2: "l"
    Output:
    Number of occurrences of "l" in "hello world" is 3
82. **Write a Java program to find the sum of all the numbers in a given string. Your program should take a string as input from the user and then find all the numbers in the string. It should then add up all the numbers and print the sum to the console.**

    Here are some requirements for your program:

    The string should be read from the console using the Scanner class.
    Your program should use regular expressions to find all the numbers in the string.
    The sum of the numbers should be calculated using a loop.
    If there are no numbers in the string, your program should print a message saying so.
    Example:

Input: "Hello 123 world 456"
Output: 579

Write test cases on the same program by using the assertion method in Junit.

83. Write a Java program to find the sum of all the numerical digits in a given string s1. The string can contain alphabets and digits in any order, but the digits are guaranteed to be consecutive. Your program should output the total sum of all the digits in the string.

    For example, given the string s1 = "abc4ui78fg95", the program should output sum = 177, since the digits in the string are 4, 7, 8, 9, and 5, and their sum is 4 + 7 + 8 + 9 + 5 = 33.
84. Given a matrix of integers, write a program to modify the matrix such that if any element of the matrix is 0, then the entire row and column of that element should be set to 0. Implement this program using Java.

    The program should take the following input:

    The matrix, represented as a two-dimensional array of integers.

    The program should perform the following tasks:

    Check for the presence of 0 in the matrix and record the positions of the 0 elements.
    Modify the matrix by setting the entire row and column to 0 if any element in the row or column contains a 0.
    The program should return the modified matrix as output.

    For example, given the matrix:

    { 1, 2, 3 }
    { 4, 0, 6 }
    { 7, 8, 9 }

    The program should modify the matrix to:

    { 1, 0, 3 }
    { 0, 0, 0 }
    { 7, 0, 9 }

    And return the modified matrix as output.
85. Write a Java program to find all the duplicate elements in an integer array with less time complexity.

    Example input:

    int[] arr = {4, 2, 4, 5, 2, 3, 1, 1, 6, 7, 7};

    Expected output:

Duplicate elements in the given array are:
4 2 1 7

Constraints:

The input array can have a maximum of 10^5 elements.
The program should have a time complexity of O(n), where n is the size of the input array.

86. Write a Java program to print even and odd numbers using multiple threads in a multithreaded environment. The program should have two threads, one for printing even numbers and the other for printing odd numbers. Each thread should print numbers from 1 to 100, and the output should be interleaved between the threads, with even numbers printed by one thread and odd numbers printed by the other.
87. Write a Java program to reverse an array without using any inbuilt methods.
88. Write a SQL query to fetch department details for a given employee ID?
89. Write a program in Java that demonstrates thread safety by accessing a shared resource concurrently from multiple threads.
90. Write a Java program to display the highest prime number.
91. Program to find the largest number in a given input array and replace it with a given number/string.
    In an array take alternative numbers and reverse them. Put it again in the place of array again e.g., arr = [1,2,3,4,5,6,7,8]

    newarr = [1,8,3,6,5,4,7,2]
92. Given an unsorted array a = [3,1,5,2,7], find alternate elements "3,5,7", reverse them "7,5,3" and place them in the original array

    Expected result [7,1,5,2,3]
93. Find second largest number [3,1,5,2,7]
94. Out of given 3 arrays with n number of elements, find out the common element(s) for all 3 arrays.
95. From a given array, make n elements rotate.

    eg. given Array:[21,54,11,35,4,18], index:3; output Array: [4,18,21,54,11,35]
96. From an array, find out the elements, and their repetition and print as element=count.

    eg.

    Input Array: [2,1,3,4,6,5,4,7,5,6,8,2,3,6,2,6,4,4,7,3,4,5,6]

    Output:

    2=3

    1=1

    3=3

4=5

6=5

5=5

7=2

8=1

97. Print the right diagonal of the matrix

    Input:

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

    Output:

    4 7 10 13
98. String= "abaade", search for a character "a" in the given string and return its occurrence.
99. How will you remove duplicate elements from an array?

Section C: Soft Skills Questions

100. Tell me about a project you worked on where you had to communicate technical information to a non-technical audience.
101. How do you prioritize and manage your workload?
102. Can you describe a time when you had to work with a difficult team member?
103. How do you handle criticism of your code or work?
104. Can you describe a time when you had to work under pressure to meet a deadline?
105. How do you ensure that your code is maintainable and scalable?
106. Tell me about a time when you had to learn a new technology or programming language quickly.
107. How do you keep up-to-date with the latest developments in software development?
108. Can you describe a time when you had to work on a project that required collaboration with multiple teams?
109. How do you approach problem-solving in your work?
110. Can you give an example of how you have applied creativity in your work as a software developer?
111. Can you describe a time when you had to work on a project with a tight budget?

112. How do you ensure that your code is properly documented?
113. Tell me about a time when you had to work with a client or end-user to understand their needs and requirements.
114. How do you approach debugging complex software issues?
115. Can you describe a time when you had to make a trade-off between code quality and speed of delivery?
116. How do you collaborate with other developers on a project?
117. Tell me about a time when you had to mentor or train other developers.
118. Can you describe a time when you had to persuade a team to adopt a new technology or approach?
119. How do you approach testing and ensuring the quality of your code?